

FhamIA

Documentation Technique

Plateforme d'apprentissage assistée par intelligence artificielle

[Installation](#) · [Architecture](#) · [Base de données](#) · [API](#) · [Sécurité](#)

Version 2.0 — 19 juillet 2026

Document technique — usage pédagogique / stage

Historique des versions

Version	Date	Auteur	Description des modifications
1.0	19/07/2026	Équipe FhamIA	Première version : installation, architecture, modèles, routes API
2.0	19/07/2026	Équipe FhamIA	Refonte complète : diagrammes d'architecture, schéma EER, fiches API détaillées, section sécurité, arborescence complète, harmonisation de la présentation

Remarque

Ce document décrit l'état réel du code source à la date indiquée ci-dessus. Toute évolution du projet (nouveaux endpoints, nouveaux modèles, changement de configuration) doit être répercutée ici et le numéro de version incrémenté en conséquence.

Sommaire

1. Introduction
2. Vue d'ensemble de l'architecture
 - 2.1 Architecture générale
 - 2.2 Architecture Backend
 - 2.3 Architecture Frontend
 - 2.4 Architecture du système RAG
 - 2.5 Flux de communication complet
3. Arborescence du projet
4. Installation
5. Base de données
6. Documentation de l'API
7. Frontend
8. Sécurité
9. Technologies et dépendances
 10. Workflow applicatif complet
 11. Conclusion technique et pistes d'évolution

1. Introduction

FhamIA est une plateforme d'apprentissage en ligne qui combine des parcours de cours structurés, des quiz de validation des connaissances, un environnement **playground** d'exercices techniques sandboxés (terminal Linux, requêtes SQL, calcul de sous-réseaux) et un tuteur pédagogique basé sur l'IA générative (architecture RAG combinée à un modèle de langage local via Ollama), avec un support spécifique de la darija marocaine.

Ce document constitue la référence technique du projet. Il s'adresse aux développeurs souhaitant installer, maintenir ou faire évoluer la plateforme, ainsi qu'à un encadrant technique souhaitant évaluer l'architecture mise en œuvre.

1.1 Périmètre du document

- Architecture logicielle globale (backend, frontend, IA)
- Procédure d'installation et de configuration de l'environnement de développement
- Modèle de données et relations entre entités
- Référence complète des endpoints de l'API REST
- Analyse de la posture de sécurité actuelle du projet
- Liste des technologies et dépendances utilisées

Remarque

Toutes les informations de ce document ont été extraites directement du code source du projet (modèles Django, vues, sérialiseurs, fichiers de configuration, package .json). Aucune fonctionnalité non implémentée n'est décrite comme existante ; les éléments incertains ou à vérifier sont explicitement signalés.

2. Vue d'ensemble de l'architecture

Le projet suit une architecture découplée **backend / frontend**, complétée par un sous-système d'intelligence artificielle (RAG + LLM local) indépendant du reste de l'application.

2.1 Architecture générale

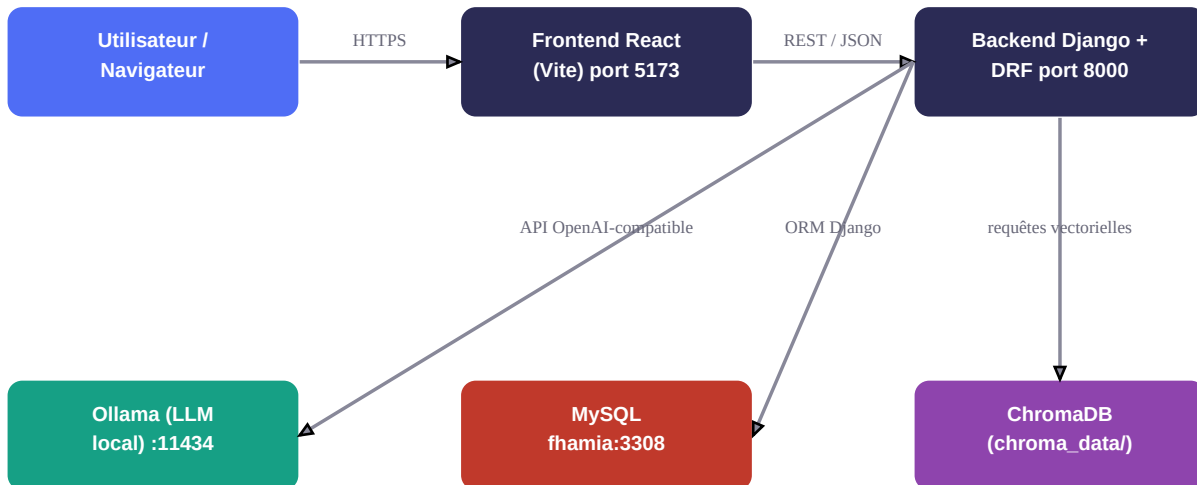


Diagramme source (Mermaid) — Architecture générale

```
graph LR
    U[Utilisateur / Navigateur] --> F[Frontend React - Vite - :5173]
    F --> B[Backend Django + DRF - :8000]
    B --> M[(MySQL - fhamia - :3308)]
    B --> C[(ChromaDB - chroma_data/)]
    B --> O[(Ollama LLM local - :11434)]
```

2.2 Architecture Backend

Le backend est un projet Django 6 organisé en applications métier indépendantes, chacune exposant ses propres routes REST via Django REST Framework.

Application Django	Responsabilité
accounts	Comptes utilisateurs, administrateurs, certificats, messages de support
courses	Parcours pédagogiques, leçons, playgrounds associés, visualisations
quiz	Quiz, questions, suivi de progression des apprenants
playground	Environnement sandboxé (shell Linux, SQL, calcul réseau)
rag	Tuteur IA : recherche vectorielle (ChromaDB) et génération de texte (Ollama)

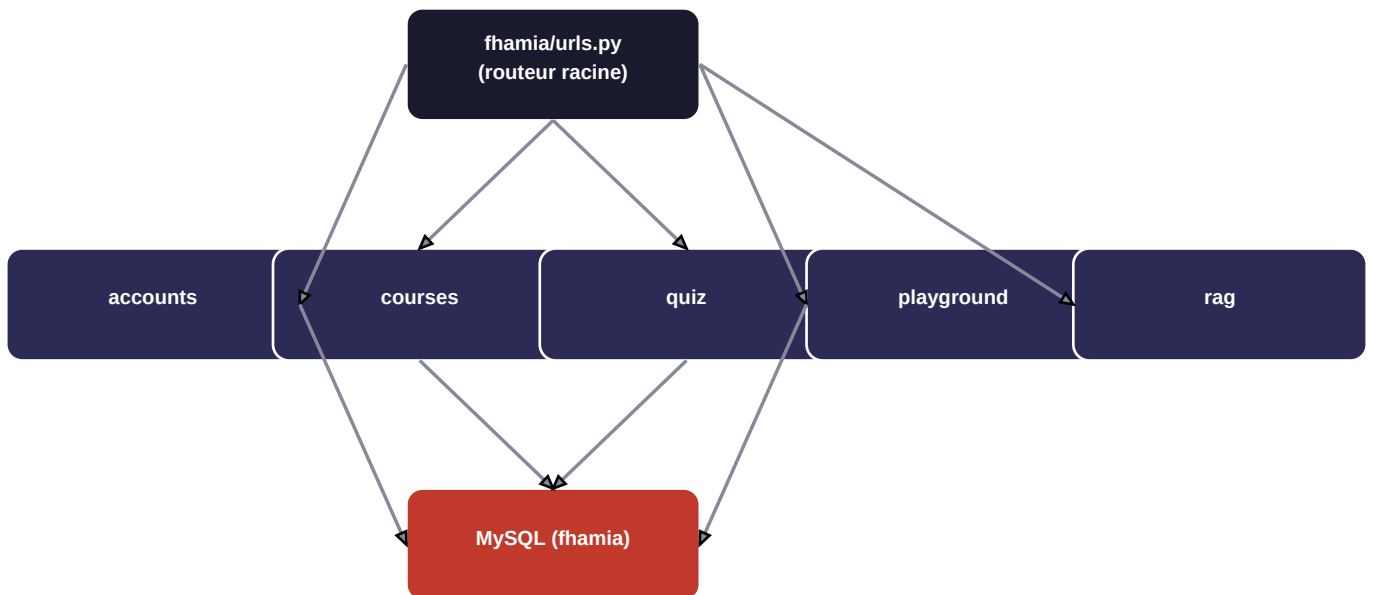


Diagramme source (Mermaid) — Architecture Backend

```

graph TD
  URLs[fhamia/urls.py] --> ACC[accounts]
  URLs --> COU[courses]
  URLs --> QUI[quiz]
  URLs --> PLA[playground]
  URLs --> RAG[rag]
  ACC --> DB["MySQL fhamia"]
  COU --> DB
  QUI --> DB
  PLA --> DB
  
```

2.3 Architecture Frontend

Le frontend est une application React 19 mono-page (SPA) construite avec Vite, structurée en pages (routes) et composants réutilisables, communiquant avec l'API via un module centralisé `api.js`.

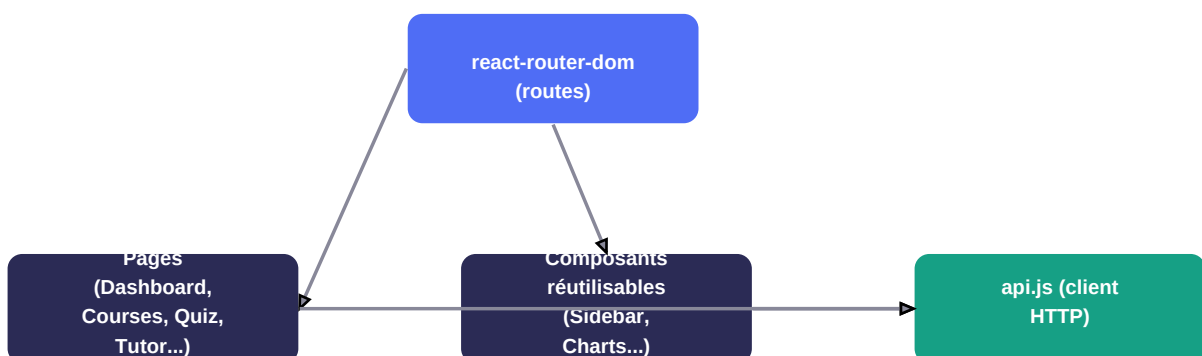


Diagramme source (Mermaid) — Architecture Frontend

```

graph TD
  R[react-router-dom] --> P[Pages]
  
```

```
R --> C[Composants]
P --> A[api.js]
C --> A
A -->|fetch REST| BACKEND[API Django]
```

2.4 Architecture du système RAG

Le tuteur IA repose sur un pipeline de type **Retrieval-Augmented Generation (RAG)** : la question de l'apprenant est utilisée pour retrouver les passages de cours les plus pertinents dans une base vectorielle (ChromaDB), puis ces passages sont injectés dans le prompt envoyé au modèle de langage (Ollama) afin de produire une réponse ancrée dans le contenu pédagogique réel.

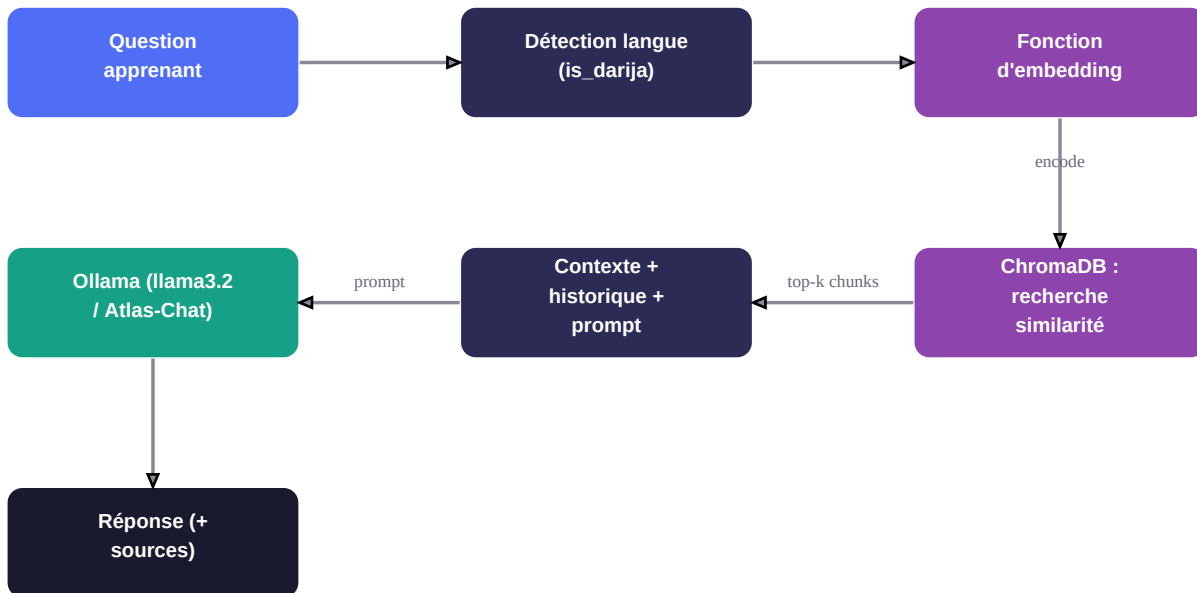


Diagramme source (Mermaid) — Pipeline RAG

```
graph LR
  U[Utilisateur] --> F[Frontend React]
  F --> B[Backend Django - rag/views.py]
  B --> D{is_darija?}
  D -->|oui| T[Traduction FR via Ollama]
  D -->|non| E
  T --> E[Embedding de la question]
  E --> C[(ChromaDB - recherche vectorielle)]
  C --> X[Récupération du contexte - top-k chunks]
  X --> P[Construction du prompt - contexte + historique]
  P --> O[Ollama - llama3.2 ou Atlas-Chat darija]
  O --> R[Réponse générée]
  R --> F
  F --> U
```

Remarque

Le pipeline exact implémenté dans `rag/services.py` : 1) détection de la darija (`is_darija`) et traduction éventuelle vers le français pour la recherche ; 2) recherche vectorielle dans ChromaDB (`collection.query`) ; 3) construction du prompt avec le contexte récupéré ; 4) génération via Ollama (`ask_ollama / ask_ollama_stream`) ; 5) traduction de la réponse en darija si nécessaire. L'accès à ChromaDB est protégé par un verrou `threading.Lock` car le client n'est pas garanti thread-safe.

2.5 Flux de communication complet

Vue combinée des échanges entre les différentes couches du système, du navigateur jusqu'au modèle de langage.

Diagramme source (Mermaid) — Flux de communication complet

```
sequenceDiagram
    participant U as Utilisateur
    participant F as Frontend React
    participant B as Backend Django/DRF
    participant M as MySQL
    participant C as ChromaDB
    participant O as Ollama

    U->>F: Interaction (navigation, question au tuteur...)
    F->>B: Requête HTTP REST (JSON)
    alt Opération standard (cours, quiz, comptes)
        B->>M: Requête ORM
        M-->>B: Données
    else Question au tuteur IA
        B->>C: Recherche vectorielle (top-k)
        C-->>B: Passages de cours pertinents
        B->>O: Prompt (contexte + question)
        O-->>B: Réponse générée (streaming ou complète)
    end
    B-->>F: Réponse JSON / flux texte
    F-->>U: Affichage du résultat
```

3. Arborescence du projet

```
FhamIA projet/
■ ■ ■ fhamia/                                # Configuration du projet Django (settings, urls, wsgi/asgi)
■   ■ ■ ■ settings.py
■   ■ ■ ■ urls.py                            # Routeur racine : /api/rag, /api/accounts, /api/courses, ...
■   ■ ■ ■ wsgi.py
■   ■ ■ ■ asgi.py                            # Présent mais non utilisé (pas de WebSocket / Channels)
■ ■ ■ accounts/                             # Utilisateurs, administrateurs, certificats, support
■   ■ ■ ■ models.py
■   ■ ■ ■ serializers.py
■   ■ ■ ■ views.py
■   ■ ■ ■ urls.py
■ ■ ■ courses/                             # Parcours, leçons, playgrounds liés, visualisations
■ ■ ■ quiz/                                # Quiz, questions, progression
■ ■ ■ playground/                          # Sandbox shell Linux / SQL / calcul de sous-réseaux
■   ■ ■ ■ linux_shell.py
■   ■ ■ ■ sql_sandbox.py
■   ■ ■ ■ subnet_calc.py
■ ■ ■ rag/                                  # Tuteur IA (RAG + LLM)
■   ■ ■ ■ chroma_client.py                  # Client ChromaDB (PersistentClient)
■   ■ ■ ■ embeddings.py                    # Fonction d'embedding (modèle multilingue)
■   ■ ■ ■ llm.py                           # Appels à Ollama (ask_ollama / ask_ollama_stream)
■   ■ ■ ■ services.py                      # Logique métier du pipeline RAG
■   ■ ■ ■ language.py                      # Détection darija / small talk
■   ■ ■ ■ views.py
■ ■ ■ frontend/                            # Application React (Vite)
■   ■ ■ ■ src/
■   ■   ■ ■ ■ App.jsx
■   ■   ■ ■ ■ api.js                        # Client HTTP centralisé vers l'API Django
■   ■   ■ ■ ■ main.jsx
■   ■   ■ ■ ■ pages/                       # Dashboard, Courses, Quiz, Tutor, AdminX, Labs...
■   ■   ■ ■ ■ components/                 # Sidebar, TopBar, Charts, PlaygroundShell...
■   ■   ■ ■ ■ data/
■   ■   ■ ■ ■ assets/
■   ■ ■ ■ package.json
■ ■ ■ chroma_data/                         # Stockage persistant de la base vectorielle ChromaDB
■ ■ ■ venv/                                # Environnement virtuel Python (non versionné)
■ ■ ■ db.sqlite3                           # Présent mais inutilisé (le projet est configuré sur MySQL)
■ ■ ■ .env                                 # Variables d'environnement (clés API, config Ollama)
■ ■ ■ requirements.txt                     # Dépendances Python
■ ■ ■ manage.py                           # Point d'entrée Django (runserver, migrate, ...)
```

Remarque

Il n'existe pas de dossier media/ ni static/ peuplé dans le projet actuel (seul STATIC_URL est configuré). Le fichier db.sqlite3 présent à la racine est vide et n'est pas utilisé : la base de données effective est MySQL (voir fhamia/settings.py).

4. Installation

4.1 Prérequis logiciels

Logiciel	Version recommandée	Usage
Python	3.11 ou supérieur	Exécution du backend Django
Django	6.0.7 (figé dans le projet)	Framework web backend
MySQL Server	8.x	Base de données relationnelle principale
Node.js	18 ou supérieur	Exécution de l'outillage frontend (Vite)
npm	9 ou supérieur	Gestion des paquets frontend
Ollama	dernière version stable	Serveur d'inférence LLM local
Git	optionnel	Gestion de versions

4.2 Installation du backend Django

```
# 1. Se placer dans le dossier du projet
cd "FhamIA projet"

# 2. Créer et activer un environnement virtuel
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # Linux / macOS

# 3. Installer les dépendances Python
pip install -r requirements.txt

# 4. Vérifier/installer les paquets DRF et CORS (utilisés dans le code
#   mais absents de requirements.txt au moment de la rédaction)
pip install djangorestframework django-cors-headers

# 5. Créer le fichier .env à la racine du projet (voir section 4.5)

# 6. Créer la base de données MySQL "fhamia"
mysql -u root -p -e "CREATE DATABASE fhamia CHARACTER SET utf8mb4;"

# 7. Générer et appliquer les migrations
python manage.py makemigrations
python manage.py migrate

# 8. Créer un superutilisateur pour l'administration Django
python manage.py createsuperuser

# 9. Lancer le serveur de développement
python manage.py runserver
# API disponible sur http://127.0.0.1:8000/api/
# Interface d'admin sur http://127.0.0.1:8000/admin/
```

Point d'attention

Les identifiants de connexion MySQL (USER='root', PASSWORD='', PORT='3308') sont actuellement codés en dur dans `fhamia/settings.py`. Adapter ces valeurs à votre environnement local, et voir la section 8 (Sécurité) pour la recommandation de les externaliser.

4.3 Lancement du sous-système IA (Ollama + ChromaDB)

```
# Démarrer le serveur Ollama
ollama serve

# Télécharger le modèle de chat généraliste par défaut
ollama pull llama3.2

# Télécharger le modèle dédié à la darija marocaine
ollama pull hf.co/QuantFactory/Atlas-Chat-2B-GGUF:Q4_K_M

# ChromaDB ne nécessite pas de service séparé : il fonctionne en mode
# "PersistentClient" et crée automatiquement le dossier chroma_data/
# à la racine du projet lors de la première indexation/requête.
```

4.4 Installation du frontend React

```
cd frontend
npm install
npm run dev
# Application disponible sur http://localhost:5173
```

4.5 Fichier de configuration .env

Le fichier `.env` doit être placé à la racine du projet (à côté de `manage.py`). Il est chargé par `python-dotenv` dans `fhamia/settings.py`. Voici les variables réellement utilisées dans le projet :

```
# --- .env ---

# Clé API OpenAI (non utilisée activement : le projet utilise le client
# openai pour dialoguer avec l'API compatible OpenAI exposée par Ollama
# en local ; peut rester vide en usage 100% local)
OPENAI_API_KEY=

# URL du serveur Ollama local
OLLAMA_BASE_URL=http://localhost:11434

# Modèle de chat généraliste utilisé par défaut
OLLAMA_MODEL=llama3.2

# --- Variables optionnelles (lues avec valeur par défaut dans settings.py,
# non définies dans le .env actuel du projet) ---

# Modèle d'embedding pour l'indexation ChromaDB
# EMBEDDING_MODEL=paraphrase-multilingual-MiniLM-L12-v2

# Modèle dédié aux réponses en darija marocaine
# DARIJA_MODEL=hf.co/QuantFactory/Atlas-Chat-2B-GGUF:Q4_K_M
```

Bonne pratique

Les paramètres `EMBEDDING_MODEL` et `DARIJA_MODEL` possèdent des valeurs par défaut dans le code (`os.environ.get(..., default)`) : il n'est pas obligatoire de les définir dans `.env` pour que l'application fonctionne, mais cela permet de les surcharger facilement selon l'environnement.

5. Base de données

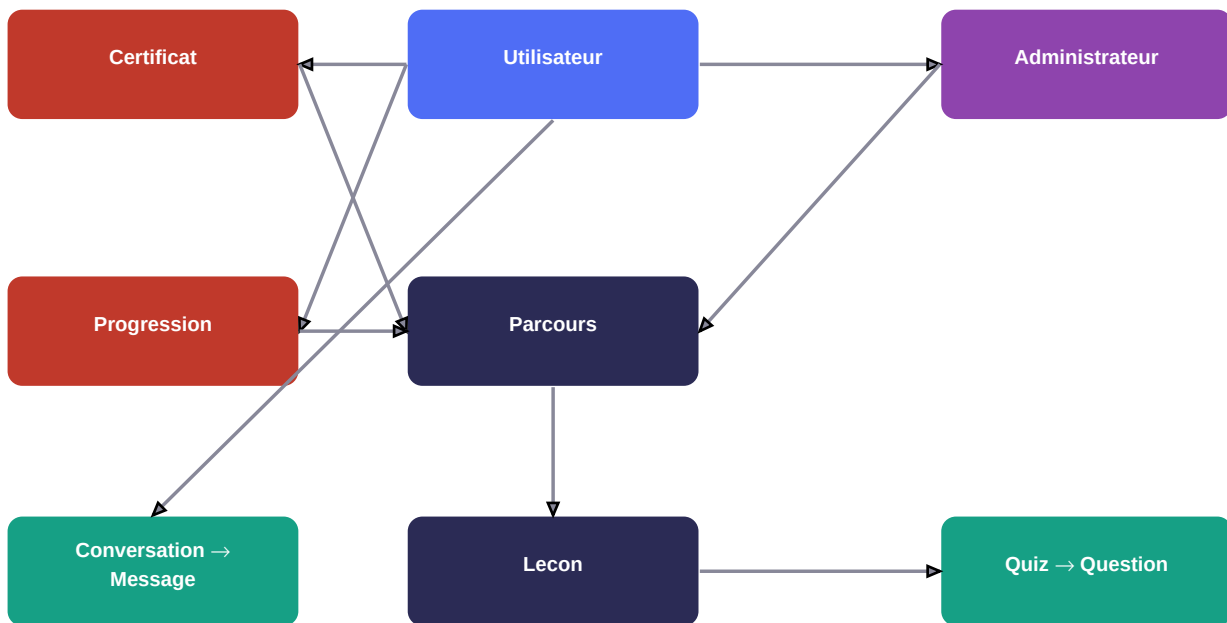
Moteur : **MySQL** (base fhamia). Le schéma est entièrement géré par le système de migrations de Django, organisé par application. Chaque modèle possède une clé primaire implicite `id` (`BigAutoField`), non répétée dans les tableaux ci-dessous sauf mention contraire.

5.1 Schéma entité-relation (EER) simplifié

Diagramme source (Mermaid) — Schéma EER

```
erDiagram
    UTILISATEUR ||--o| ADMINISTRATEUR : "héritage (multi-table)"
    UTILISATEUR ||--o{ CERTIFICAT : possede
    UTILISATEUR ||--o{ PROGRESSION : suit
    UTILISATEUR ||--o{ FICHIER_VIRTUEL : possede
    UTILISATEUR ||--o{ CONVERSATION : initie
    UTILISATEUR }o--o{ PARCOURS : "apprenants (M2M)"
    ADMINISTRATEUR ||--o{ PARCOURS : gere
    PARCOURS ||--o{ CERTIFICAT : concerne
    PARCOURS ||--o{ PROGRESSION : concerne
    PARCOURS ||--o{ LECON : contient
    LECON ||--o{ PLAYGROUND : contient
    LECON ||--o{ VISUALISATION : contient
    LECON ||--o| QUIZ : "possede (1-1)"
    QUIZ ||--o{ QUESTION : contient
    TUTEUR_IA ||--o{ CONVERSATION : anime
    CONVERSATION ||--o{ MESSAGE : contient

    UTILISATEUR {
        int id PK
        string email UK
        string mot_de_passe
        datetime date_inscription
        string etat_compte
    }
    ADMINISTRATEUR {
        int utilisateur_ptr_id PK_FK
        int niveau_acces
        date date_nomination
    }
    PARCOURS {
        int id PK
        string slug UK
        string titre
        string niveau
        int administrateur_id FK
    }
    CERTIFICAT {
        int id PK
        int utilisateur_id FK
        int parcours_id FK
        string code_verification UK
    }
```



5.2 Application accounts

Modèle	Champs clés	Contraintes
Utilisateur	prenom, nom, email, mot_de_passe (hashé), date_inscription, etat_compte	email : UNIQUE
Administrateur	hérite de Utilisateur (héritage multi-table Django) ; niveau_acces, date_nomination	PK = FK vers Utilisateur (utilisateur_ptr_id)
MessageSupport	nom, email, sujet, message, date_envoi, traite	—
Certificat	utilisateur (FK), parcours (FK), date_obtention, code_verification	code_verification : UNIQUE ; couple (utilisateur, parcours) : UNIQUE

Explication des relations

- **Administrateur hérite de Utilisateur** : Django implémente cet héritage via une table séparée liée par une clé primaire = clé étrangère (*Multi-Table Inheritance*). Un administrateur est donc aussi un utilisateur à part entière.
- **Certificat** relie un *Utilisateur* à un *Parcours* (courses) : la contrainte d'unicité empêche la délivrance de plusieurs certificats pour le même couple utilisateur/parcours.

5.3 Application courses

Modèle	Champs clés	Contraintes
Parcours	slug, titre, description, niveau, administrateur (FK, SET_NULL), apprenants (M2M vers Utilisateur)	slug : UNIQUE
Lecon	parcours (FK, CASCADE), titre, contenu, duree	—
Playground	lecon (FK, CASCADE), nom, description	—
Visualisation	lecon (FK, CASCADE), type, description	—

Explication des relations

- Un *Parcours* est géré par un *Administrateur* (relation optionnelle : suppression de l'administrateur → SET_NULL) et suivi par plusieurs *Utilisateurs* via la relation many-to-many apprenants.
- La suppression d'un *Parcours* entraîne la suppression en cascade de ses *Leçons*, et la suppression d'une *Leçon* entraîne celle de ses *Playgrounds* et *Visualisations*.

5.4 Application quiz

Modèle	Champs clés	Contraintes
Quiz	lecon (relation 1-1, CASCADE), titre, score_minimum	lecon : UNIQUE (OneToOne)
Question	quiz (FK, CASCADE), enonce, choix (JSON), reponse_correcte	—
Progression	utilisateur (FK, CASCADE), parcours (FK, CASCADE), pourcentage, points, niveau	couple (utilisateur, parcours) : UNIQUE

Explication des relations

- Chaque *Leçon* peut posséder au plus un *Quiz* (relation strictement 1-1).
- *Progression* centralise l'avancement d'un utilisateur sur un parcours (pourcentage, points, niveau) ; l'unicité du couple empêche les doublons de suivi.

5.5 Application playground

Modèle	Champs clés	Contraintes
FichierVirtuel	utilisateur (FK, CASCADE), chemin, type (dir/file), contenu, date_creation	couple (utilisateur, chemin) : UNIQUE

Ce modèle sert de système de fichiers virtuel persistant en base de données, propre à chaque utilisateur, pour les exercices de terminal Linux sandboxé.

5.6 Application rag

Modèle	Champs clés	Contraintes
TuteurIA	modele_llm, version, langue	—
Conversation	utilisateur (FK, CASCADE), tuteur (FK, SET_NULL), date_creation, statut	—
Message	conversation (FK, CASCADE), date_heure, contenu, type (utilisateur/ia)	—

Remarque

Les modèles *Tuteur IA*, *Conversation* et *Message* existent dans le schéma de base de données, mais les vues actuelles de `rag/views.py` (`query`, `query-stream`, etc.) ne persistent pas systématiquement les échanges dans ces tables — à vérifier/compléter selon les besoins d'historisation du projet.

6. Documentation de l'API

Toutes les routes sont préfixées par `/api/`. Le format d'échange est JSON (sauf les routes de streaming, qui renvoient un flux texte brut). Le routeur racine est défini dans `fhamia/urls.py`.

Point d'attention

`REST_FRAMEWORK[ 'DEFAULT_PERMISSION_CLASSES' ] = []` dans `settings.py` : aucune authentification ni autorisation n'est imposée par défaut sur l'API. Tous les endpoints décrits ci-dessous sont, en l'état, accessibles sans jeton ni session. Voir la section 8 (Sécurité).

6.1 Module accounts — `/api/accounts/`

POST `/api/accounts/register/`

Description : Crée un nouveau compte utilisateur.

Rôle fonctionnel : Inscription d'un apprenant sur la plateforme.

Exemple de requête (JSON) :

```
{
  "prenom": "Sara",
  "nom": "El Amrani",
  "email": "sara@example.com",
  "password": "MotDePasse123"
}
```

Exemple de réponse (JSON) :

```
{
  "id": 12,
  "prenom": "Sara",
  "nom": "El Amrani",
  "email": "sara@example.com",
  "date_inscription": "2026-07-19T10:00:00Z",
  "etat_compte": "actif",
  "is_admin": false
}
```

Code HTTP	Signification
201	Compte créé avec succès
400	Champ manquant ou email déjà utilisé

POST `/api/accounts/login/`

Description : Authentifie un utilisateur par email et mot de passe.

Rôle fonctionnel : Connexion (vérification via `django.contrib.auth.hashers.check_password`, sans génération de jeton de session ni JWT).

Exemple de requête (JSON) :

```
{
  "email": "sara@example.com",
  "password": "MotDePasse123"
}
```


Exemple de réponse (JSON) :

```
{
  "id": 12,
  "prenom": "Sara",
  "nom": "El Amrani",
  "email": "sara@example.com",
  "is_admin": false
}
```

Code HTTP	Signification
200	Authentification réussie
401	Email ou mot de passe incorrect

POST /api/accounts/obtenir-certificat/

Description : Délivre un certificat de fin de parcours à un utilisateur ayant atteint 100% de progression.

Rôle fonctionnel : Génération d'un certificat vérifiable (code unique).

Exemple de requête (JSON) :

```
{
  "utilisateur": 12,
  "parcours": 3
}
```

Exemple de réponse (JSON) :

```
{
  "id": 5,
  "utilisateur": 12,
  "parcours": 3,
  "code_verification": "FH-A1B2C3D4E5",
  "date_obtention": "2026-07-19T10:00:00Z"
}
```

Code HTTP	Signification
200	Certificat déjà existant, renvoyé
201	Certificat créé
400	Progression incomplète (< 100%)
404	Aucune progression trouvée

CRUD /api/accounts/utilisateurs/

Description : ViewSet DRF standard (GET liste/détail, POST, PUT/PATCH, DELETE) pour la gestion des utilisateurs.

Rôle fonctionnel : Administration des comptes utilisateurs.

Code HTTP	Signification
200	Succès (GET/PUT/PATCH)
201	Création (POST)
204	Suppression (DELETE)

Code HTTP	Signification
404	Ressource introuvable

CRUD	/api/accounts/administrateurs/
Description : ViewSet DRF standard pour la gestion des comptes administrateurs.	
Rôle fonctionnel : Administration des comptes à privilèges élevés.	

CRUD	/api/accounts/certificats/
Description : ViewSet DRF standard pour la consultation/gestion des certificats délivrés.	
Rôle fonctionnel : Suivi et vérification des certificats.	

CRUD	/api/accounts/messages-support/
Description : ViewSet DRF standard pour les messages envoyés au support.	
Rôle fonctionnel : Gestion des demandes de support utilisateur.	

6.2 Module courses — /api/courses/

CRUD	/api/courses/parcours/
Description : ViewSet DRF sur les parcours pédagogiques. Le champ de lookup est le slug (généralisé automatiquement à la création à partir du titre, avec suffixe numérique en cas de doublon).	
Rôle fonctionnel : Gestion des parcours (création, consultation, mise à jour, suppression).	

Exemple de réponse (JSON) :

```
{
  "id": 3,
  "slug": "reseaux-informatiques",
  "titre": "Réseaux informatiques",
  "niveau": "débutant",
  "administrateur": 1,
  "apprenants": [12, 15],
  "lecons": [ ... ]
}
```

POST	/api/courses/parcours/{slug}/suivre/
Description : Action personnalisée DRF (@action) : inscrit un utilisateur comme apprenant d'un parcours.	
Rôle fonctionnel : Inscription (opt-in) d'un apprenant à un parcours.	

Exemple de requête (JSON) :

```
{
  "utilisateur": 12
}
```

Exemple de réponse (JSON) :

```
{
  "status": "ok"
}
```

Code HTTP	Signification
200	Inscription effectuée
400	Champ utilisateur manquant
404	Parcours ou utilisateur introuvable

CRUD /api/courses/lecons/

Description : ViewSet DRF standard pour les leçons.

Rôle fonctionnel : Gestion du contenu pédagogique détaillé d'un parcours.

CRUD /api/courses/playgrounds/

Description : ViewSet DRF standard pour les playgrounds liés aux leçons.

Rôle fonctionnel : Association d'exercices pratiques à une leçon.

CRUD /api/courses/visualisations/

Description : ViewSet DRF standard pour les visualisations pédagogiques.

Rôle fonctionnel : Association de contenus visuels/3D à une leçon.

6.3 Module quiz — /api/quiz/

CRUD /api/quiz/quiz/

Description : ViewSet DRF standard pour les quiz.

Rôle fonctionnel : Gestion des quiz.

CRUD /api/quiz/questions/

Description : ViewSet DRF standard pour les questions de quiz.

Rôle fonctionnel : Gestion des questions et de leurs choix (champ JSON).

CRUD /api/quiz/progressions/

Description : ViewSet DRF standard pour le suivi de progression.

Rôle fonctionnel : Consultation/administration directe des progressions.

GET**/api/quiz/quiz-by-slug/****Description** : Récupère le quiz associé à un parcours via son slug (lecon__parcours__slug).**Rôle fonctionnel** : Point d'entrée utilisé par le frontend pour charger le quiz d'un parcours donné.**Exemple de réponse (JSON) :**

```
{
  "id": 2,
  "titre": "Quiz - Bases des réseaux",
  "score_minimum": 50,
  "questions": [ ... ],
  "parcours_slug": "reseaux-informatiques",
  "parcours_id": 3
}
```

Code HTTP**Signification**

200

Quiz trouvé

404

Aucun quiz pour ce parcours

POST**/api/quiz/submit-progression/****Description** : Crée ou met à jour la progression d'un utilisateur sur un parcours (upsert).**Rôle fonctionnel** : Enregistrement du résultat d'un quiz / de l'avancement d'un apprenant.**Exemple de requête (JSON) :**

```
{
  "utilisateur": 12,
  "parcours": 3,
  "pourcentage": 80,
  "points": 40
}
```

Exemple de réponse (JSON) :

```
{
  "id": 7,
  "utilisateur": 12,
  "parcours": 3,
  "pourcentage": 80,
  "points": 40,
  "niveau": 1
}
```

Code HTTP**Signification**

200

Progression créée ou mise à jour

400

Champs manquants

404

Parcours introuvable

6.4 Module playground — /api/playground/

POST**/api/playground/linux/run/**

Description : Exécute une commande dans un terminal Linux sandboxé, propre à l'utilisateur (basé sur le modèle FichierVirtual).

Rôle fonctionnel : Exercice interactif de manipulation shell.

Exemple de requête (JSON) :

```
{
  "utilisateur": 12,
  "command": "ls -la",
  "cwd": "/home/user"
}
```

Code HTTP	Signification
200	Commande exécutée (résultat dans le corps)
400	utilisateur manquant
404	Utilisateur introuvable

POST**/api/playground/sql/execute/**

Description : Exécute une requête SQL dans un bac à sable isolé par utilisateur.

Rôle fonctionnel : Exercice interactif SQL.

Exemple de requête (JSON) :

```
{
  "utilisateur": 12,
  "sql": "SELECT * FROM etudiants;"
}
```

Code HTTP	Signification
200	Requête exécutée
400	Champs manquants ou erreur SQL

POST**/api/playground/sql/reset/**

Description : Réinitialise le bac à sable SQL de l'utilisateur à son état initial.

Rôle fonctionnel : Remise à zéro d'un exercice SQL.

Exemple de requête (JSON) :

```
{
  "utilisateur": 12
}
```

Exemple de réponse (JSON) :

```
{
  "status": "ok"
}
```

Code HTTP	Signification
200	Sandbox réinitialisée
400	utilisateur manquant

POST	/api/playground/reseaux/subnet/
Description : Calcule les informations d'un sous-réseau à partir d'une notation CIDR.	
Rôle fonctionnel : Outil pédagogique de calcul de sous-réseaux (exercice réseaux).	

Exemple de requête (JSON) :

```
{
  "cidr": "192.168.1.10/26"
}
```

Code HTTP	Signification
200	Calcul effectué
400	CIDR manquant ou invalide

6.5 Module rag — /api/rag/ (tuteur IA)

Les vues de ce module sont des vues Django classiques (pas des ViewSets DRF), exposées en @csrf_exempt + @require_POST, et lisent le corps de la requête en JSON brut.

POST	/api/rag/query/
Description : Requête RAG complète (non-streaming) : recherche vectorielle + génération de réponse.	
Rôle fonctionnel : Poser une question au tuteur IA et recevoir une réponse complète en une seule fois.	

Exemple de requête (JSON) :

```
{
  "question": "C'est quoi une adresse IP ?"
}
```

Exemple de réponse (JSON) :

```
{
  "answer": "Une adresse IP est ...",
  "sources": ["cours_reseaux.pdf"],
  "question_traduite": null
}
```

Code HTTP	Signification
200	Réponse générée
400	Champ question manquant / JSON invalide
500	Erreur côté ChromaDB ou Ollama

POST	/api/rag/query-stream/
------	------------------------

Description : Variante en streaming (StreamingHttpResponse) de la requête RAG, avec prise en charge d'un historique de conversation court (2 derniers tours) et détection automatique de la darija.

Rôle fonctionnel : Chat conversationnel en temps réel avec le tuteur IA (réponse token par token).

Exemple de requête (JSON) :

```
{
  "question": "wach momkin tfsr liya IP address",
  "history": [
    {"role": "user", "text": "salam"}
  ]
}
```

Exemple de réponse (JSON) :

Flux texte brut (content-type: text/plain). En-têtes de réponse : X-Sources (sources séparées par virgule), X-Question-Traduite.

Code HTTP	Signification
200	Flux de réponse démarré
400	Champ question manquant
500	Erreur pendant la recherche vectorielle

POST **/api/rag/evaluate/**

Description : Fait évaluer par le LLM la réponse d'un apprenant à une consigne d'exercice.

Rôle fonctionnel : Correction pédagogique automatisée (feedback bienveillant en 3 à 5 phrases).

Exemple de requête (JSON) :

```
{
  "consigne": "Écrivez une requête SELECT filtrant sur l'âge.",
  "reponse": "SELECT * FROM users WHERE age > 18;"
}
```

Exemple de réponse (JSON) :

```
{
  "feedback": "Votre requête est correcte ..."
}
```

Code HTTP	Signification
200	Feedback généré
400	Champs consigne/reponse manquants
500	Erreur d'appel au LLM

POST **/api/rag/prompt/**

Description : Appel direct au LLM (Ollama), sans étape de recherche documentaire (pas de RAG).

Rôle fonctionnel : Accès bas niveau au modèle de langage, par exemple pour des outils expérimentaux (page AiLab/PromptLab du frontend).

Exemple de requête (JSON) :

```
{
  "prompt": "Explique la récursivité simplement.",
  "temperature": 0.7,
  "num_predict": 200
}
```

Exemple de réponse (JSON) :

```
{
  "response": "La récursivité est ..."
```

Code HTTP	Signification
200	Réponse générée
400	Prompt manquant ou paramètres invalides
500	Erreur d'appel au LLM

POST

/api/rag/prompt-stream/

Description : Variante en streaming de l'appel direct au LLM.

Rôle fonctionnel : Même usage que /prompt/ mais avec réponse progressive (token par token).

Exemple de requête (JSON) :

```
{
  "prompt": "Écris un pome sur Python.",
  "num_predict": 300
}
```

Exemple de réponse (JSON) :

Flux texte brut (content-type: text/plain)

Code HTTP	Signification
200	Flux démarré
400	Prompt manquant ou paramètres invalides

7. Frontend

Le frontend est une application **React 19** propulsée par **Vite 8**, située dans le dossier `frontend/`. Le routage est géré par `react-router-dom` (v7).

7.1 Structure des dossiers

```
frontend/src/
■■■ App.jsx           # Composant racine, déclaration des routes
■■■ main.jsx          # Point d'entrée (ReactDOM.createRoot)
■■■ api.js            # Client HTTP centralisé vers le backend Django
■■■ index.css
■■■ pages/            # Une page par route (32 fichiers)
■■■ components/       # Composants réutilisables (Sidebar, TopBar, Charts...)
■■■ data/             # Données statiques / mock (courses, quizzes, visualisations)
■■■ utils/
■■■ assets/           # Images (hero, support, logos)
```

7.2 Pages principales

Catégorie	Pages (fichiers dans src/pages/)
Authentification	Login, Register
Espace apprenant	Dashboard, Courses, CourseDetail, DomainCourses, Reader, Quiz, QuizList, Certificate, Profile
Playgrounds techniques	LinuxLab, SqlLab, SubnetLab, PlaygroundShell (composant)
Tuteur IA	Tutor, AiLab, AiLabSelect, PromptLab
Visualisations	VisualisationSelect, VisualisationDetail
Administration	AdminDashboard, AdminUsers, AdminPaths, AdminLessons, AdminQuizManagement, AdminCertificates, AdminStats
Divers	Landing, Support, Terms

7.3 Composants notables

- **Sidebar / TopBar / AdminSidebar / AdminHeader** — navigation générale et espace admin
- **AdminRoute** — composant de garde de route pour l'espace administrateur
- **DonutChart / LineChart** — visualisations de statistiques (tableau de bord admin)
- **PlaygroundShell** — terminal interactif consommant l'API playground
- **components/visualizations** — rendus 3D basés sur `@react-three/fiber` et `@react-three/drei`
- **Starfield / Logo** — éléments décoratifs de l'interface

7.4 Communication avec l'API

Toutes les requêtes vers le backend transitent par `src/api.js`, qui centralise la construction des appels HTTP vers `/api/...`. Le CORS est autorisé côté Django pour les ports de développement Vite (5173 et 5174).

Remarque

Les bibliothèques `pdfjs-dist`, `jspdf` et `html2canvas` sont utilisées pour l'affichage et l'export de documents PDF côté client (ex. génération visuelle du certificat de fin de parcours).

■ Capture d'écran à insérer

interface du tableau de bord apprenant (Dashboard.jsx)

■ Capture d'écran à insérer

interface du tableau de bord administrateur (AdminDashboard.jsx)

■ Capture d'écran à insérer

playground / terminal Linux sandboxé (LinuxLab.jsx)

■ Capture d'écran à insérer

interface du tuteur IA en conversation (Tutor.jsx)

8. Sécurité

Cette section présente l'état réel des mécanismes de sécurité implémentés dans le projet, tels qu'observés dans le code source, ainsi que les points nécessitant une attention avant toute mise en production.

8.1 Authentification

L'authentification est **artisanale** : la vue `login_view` (`accounts/views.py`) vérifie l'email et le mot de passe haché via `django.contrib.auth.hashers.check_password`, mais ne génère **ni session Django, ni jeton JWT, ni cookie d'authentification**. Le frontend reçoit uniquement les informations de l'utilisateur en réponse.

Point d'attention

Aucun mécanisme de type JWT (`django-rest-framework-simplejwt`) ou de session DRF n'est présent dans le projet à ce jour. Les requêtes suivantes ne sont donc protégées par aucun jeton : le frontend doit lui-même empêcher l'accès aux pages sensibles (ex. `AdminRoute`), mais rien n'empêche un appel direct à l'API sans passer par l'interface.

8.2 Autorisation

`REST_FRAMEWORK['DEFAULT_PERMISSION_CLASSES'] = []` : DRF n'applique **aucune permission par défaut**. Tous les ViewSets (utilisateurs, administrateurs, parcours, quiz, certificats, etc.) sont donc accessibles en lecture et en écriture sans restriction de rôle.

Point d'attention

La distinction apprenant / administrateur n'existe côté backend que via l'existence d'une ligne dans la table `Administrateur` (héritage de `Utilisateur`) et le champ calculé `is_admin` du sérialiseur. Aucune vérification de ce statut n'est effectuée côté API avant d'autoriser une opération sensible (ex. création d'un parcours, consultation de tous les utilisateurs).

8.3 Validation des données

- Les sérialiseurs DRF (`ModelSerializer`) assurent une validation de base (types, champs requis, unicité déclarée sur les modèles comme email ou slug).
- Les vues fonction (`@api_view`) implémentent des contrôles manuels explicites (champs requis, existence des objets liés) avec renvoi de codes 400/404 appropriés.
- Les vues du module `rag` valident le JSON reçu (`json.loads` avec gestion d'exception) et rejettent les questions/prompts vides.

8.4 CORS (Cross-Origin Resource Sharing)

Géré par `django-cors-headers`. `CORS_ALLOWED_ORIGINS` restreint les origines autorisées aux ports de développement Vite locaux (`localhost:5173`, `127.0.0.1:5173`, `:5174`). `CORS_EXPOSE_HEADERS` expose les en-têtes personnalisés `X-Sources` et `X-Question-Traduite` utilisés par le streaming RAG.

Bonne pratique

Cette configuration est adaptée au développement local. Avant un déploiement, il faudra ajouter le(s) domaine(s) de production réel(s) à `CORS_ALLOWED_ORIGINS`.

8.5 Variables d'environnement et secrets

Point d'attention

SECRET_KEY Django est codée en dur dans `settings.py` (préfixe `django-insecure-`, valeur par défaut générée par `startproject`) et `DEBUG = True` est actif en permanence. Les identifiants MySQL (utilisateur `root`, mot de passe vide) sont également en dur dans le fichier. Ces trois éléments doivent impérativement être externalisés via `.env` et `DEBUG` désactivé avant toute mise en production.

Seules les variables liées à Ollama/OpenAI (`OPENAI_API_KEY`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`) transitent actuellement par le fichier `.env` via `python-dotenv`.

8.6 Protection des mots de passe

Les mots de passe utilisateurs sont hachés via `django.contrib.auth.hashers.make_password / check_password` (algorithme PBKDF2 par défaut de Django) avant stockage dans le champ `mot_de_passe`. Les validateurs standard de Django (`AUTH_PASSWORD_VALIDATORS` : similarité avec l'utilisateur, longueur minimale, mots de passe courants, mot de passe entièrement numérique) sont déclarés dans `settings.py`.

Remarque

Ces validateurs standard s'appliquent aux formulaires natifs de Django (ex. `createsuperuser`) mais ne sont pas explicitement appelés dans la vue `register_view` personnalisée — à vérifier si un renforcement côté API est souhaité.

8.7 Sandbox du playground

Point d'attention

Les endpoints `/api/playground/linux/run/` et `/api/playground/sql/execute/` exécutent des commandes/requêtes fournies par l'utilisateur final. Le niveau réel d'isolation (simulation applicative pure vs. exécution système réelle) doit être vérifié dans `linux_shell.py` et `sql_sandbox.py` afin d'exclure tout risque d'évasion de bac à sable ou d'injection vers le système hôte.

8.8 Synthèse et recommandations

Thème	État actuel	Recommandation
Authentification API	Absente (pas de session/JWT)	Ajouter <code>django-rest-framework-simplejwt</code> ou l'authentification par session DRF
Permissions DRF	<code>DEFAULT_PERMISSION_CLASSES</code> vide	Définir <code>IsAuthenticated</code> par défaut + permissions par rôle (admin/apprenant)
SECRET_KEY / DEBUG	En dur, <code>DEBUG=True</code>	Externaliser via <code>.env</code> , <code>DEBUG=False</code> en production
Identifiants MySQL	En dur (root, mot de passe vide)	Externaliser via <code>.env</code>
Mots de passe	Hachés (PBKDF2 Django)	Conforme — maintenir cette pratique
CORS	Restreint aux ports de dev locaux	Ajouter les domaines de production au déploiement
Sandbox playground	Isolation à confirmer	Auditer <code>linux_shell.py</code> / <code>sql_sandbox.py</code>

9. Technologies et dépendances

9.1 Stack technique

Technologie	Version	Utilisation
Python	3.11+	Langage du backend
Django	6.0.7	Framework web backend, ORM, admin
Django REST Framework	à vérifier (non pinné dans requirements.txt)	Construction de l'API REST (ViewSets, serializers, routers)
django-cors-headers	à vérifier (non pinné dans requirements.txt)	Gestion des en-têtes CORS
MySQL	8.x (mysqlclient 2.2.8)	Base de données relationnelle principale
ChromaDB	1.5.9	Base vectorielle pour la recherche sémantique (RAG)
Ollama	service externe local	Serveur d'inférence pour les modèles de langage (llama3.2, Atlas-Chat)
openai (SDK Python)	2.45.0	Client utilisé pour dialoguer avec l'API compatible OpenAI exposée par Ollama
pypdf	6.14.2	Traitement de documents PDF côté backend (ingénierie des contenus/RAG)
python-dotenv	1.2.2	Chargement des variables d'environnement (.env)
React	19.2.7	Bibliothèque UI du frontend
Vite	8.1.1	Outil de build / serveur de développement frontend
react-router-dom	7.18.1	Routage côté client
@react-three/fiber, @react-three/drei, three	9.6.1 / 10.7.7 / 0.185.1	Rendu 3D pour les visualisations pédagogiques
pdfjs-dist, jspdf, html2canvas	4.10.38 / 4.2.1 / 1.4.1	Affichage et export PDF côté client (ex. certificats)
lucide-react	1.24.0	Bibliothèque d'icônes
JWT	non utilisé	Aucune bibliothèque JWT présente dans le projet à ce jour

Remarque

Django REST Framework et django-cors-headers sont réellement utilisés dans le code (`rest_framework` et `corsheaders` dans `INSTALLED_APPS`) mais n'apparaissent pas explicitement dans `requirements.txt` au moment de la rédaction. Exécuter `pip freeze` dans l'environnement de développement pour confirmer les versions installées et compléter le fichier.

9.2 Principales dépendances backend (requirements.txt)

Extrait des paquets les plus significatifs (liste complète dans `requirements.txt`, 89 paquets au total, incluant les dépendances transitives de ChromaDB comme `grpcio`, `opentelemetry-*`, `onnxruntime`, `kubernetes`) :

```
Django==6.0.7
mysqlclient==2.2.8
chromadb==1.5.9
openai==2.45.0
huggingface_hub==1.23.0
tokenizers==0.23.1
pypdf==6.14.2
python-dotenv==1.2.2
numpy==2.5.1
onnxruntime==1.27.0
```

9.3 Principales dépendances frontend (package.json)

```
react: ^19.2.7      react-dom: ^19.2.7
react-router-dom: ^7.18.1 vite: ^8.1.1
@react-three/fiber: ^9.6.1 @react-three/drei: ^10.7.7
three: ^0.185.1      lucide-react: ^1.24.0
pdfjs-dist: ^4.10.38 jspdf: ^4.2.1
html2canvas: ^1.4.1  eslint: ^10.6.0 (dev)
```

10. Workflow applicatif complet

Parcours type d'un apprenant à travers la plateforme, de la connexion jusqu'à l'obtention d'un certificat.

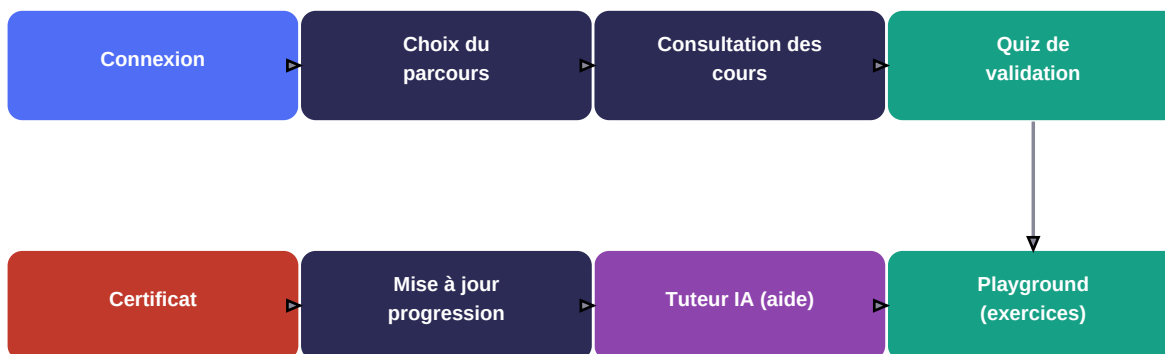


Diagramme source (Mermaid) — Workflow applicatif

```
graph LR
  A[Connexion - /api/accounts/login] --> B[Choix du parcours - /api/courses/parcours]
  B --> C[Inscription au parcours - /parcours/{slug}/suivre]
  C --> D[Consultation des leçons]
  D --> E[Quiz de validation - /api/quiz/quiz-by-slug]
  E --> F[Soumission des résultats - /api/quiz/submit-progression]
  D --> G[Playground - exercices pratiques]
  D --> H[Tuteur IA - /api/rag/query-stream]
  F --> I[Mise à jour de la progression]
  I --> J{Progression = 100% ?}
  J -->|oui| K[Obtention du certificat - /api/accounts/obtenir-certificat]
  J -->|non| D
```

À chaque étape, l'apprenant peut solliciter le tuteur IA (module *rag*) pour obtenir des explications complémentaires, ou s'exercer dans le playground (terminal Linux, SQL, calcul de sous-réseaux) lié à la leçon en cours.

■ Capture d'écran à insérer

parcours utilisateur complet, de la connexion au certificat

11. Conclusion technique et pistes d'évolution

FhamIA implémente une architecture clé en main pour une plateforme d'apprentissage augmentée par IA : un backend Django/DRF modulaire (cinq applications métier clairement séparées), une base de données relationnelle classique pour les entités pédagogiques, et un pipeline RAG complet (ChromaDB + Ollama) pour le tuteur IA, avec une attention particulière portée au support de la darija marocaine — un choix technique différenciant pour le contexte marocain.

Le frontend React/Vite couvre l'ensemble des parcours utilisateur (apprenant et administrateur), avec des intégrations avancées (rendu 3D, export PDF) qui dépassent le périmètre d'une simple application CRUD.

11.1 Points forts

- Séparation claire des responsabilités entre applications Django
- Pipeline RAG fonctionnel avec gestion explicite du multilinguisme (français / darija)
- Modèle de données cohérent, avec contraintes d'unicité pertinentes (certificats, progressions, fichiers virtuels)
- Fonctionnalités pédagogiques variées (quiz, sandbox technique, visualisations 3D, certification)

11.2 Axes d'amélioration prioritaires

- Mettre en place une authentification robuste (JWT ou session) et des permissions DRF par rôle
- Externaliser SECRET_KEY, les identifiants MySQL et désactiver DEBUG avant toute mise en production
- Compléter `requirements.txt` avec toutes les dépendances réellement installées (DRF, cors-headers)
- Auditer l'isolation réelle du sandbox playground (shell et SQL)
- Exploiter pleinement les modèles TuteurIA / Conversation / Message pour historiser les échanges avec le tuteur IA
- Documenter l'API via un schéma OpenAPI/Swagger généré automatiquement (ex. drf-spectacular) pour garder cette documentation synchronisée avec le code

Remarque

Ce document doit être relu et mis à jour à chaque évolution significative du schéma de données, des routes API ou de la configuration de sécurité.